



Tecnicatura Universitaria
en Programación

METODOLOGÍA DE SISTEMAS I

Unidad Temática N.º 3:
U.M.L. – P.U.D.

Material Teórico
2º Año – 4º Cuatrimestre

Índice

Historia del UML 2

Modelos con UML.....	4
¿Qué es un modelo?	4
¿Cómo modelamos?	5
Propósito de UML.....	6
Bloques de construcción de UML.....	7
1. Elementos de UML	7
2. Relaciones de UML	9
3. Diagramas de UML	11
El modelo de vistas 4+ 1	12

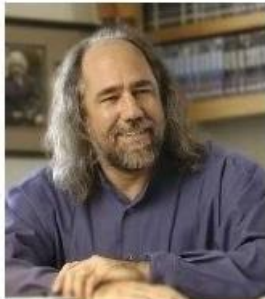
U.M.L y P.U.D 15

¿Qué es un flujo de trabajo?	16
Modelo de dominio	16
Modelo del dominio: Construcción del Diagrama de Clases	18
Modelo del dominio: Construcción del Diagrama de Casos de uso.....	25
Flujo de análisis.....	31
Diagrama de interacción	31
Diagrama de colaboración	31
Flujo de diseño	33
Diagrama de secuencia	33
Modelo de diseño: Construcción del Diagrama de Estado	33
Diagrama de actividad	35
Flujo de implementación	36
Diagrama de componentes.....	36
Diagrama de despliegue.	38

Bibliografía 41

Historia del UML

The three amigos



Grady Booch



James Rumbaugh



Ivar Jacobson

Figura (1) Elaboración propia

El lenguaje UML comenzó a gestarse en octubre de 1994 [1], cuando Rumbaugh se unió a la compañía Rational fundada por Booch (dos importantes investigadores en el área de metodología del software). El objetivo de ambos era unificar dos métodos que habían desarrollado: el método Booch y el OMT (Object Modeling Tool).

El primer borrador apareció en octubre de 1995. En esa misma época otro investigador, Jacobson, se unió a Rational y se incluyeron ideas suyas.

Estas tres personas son conocidas como los “tres amigos”. Además, este lenguaje se abrió a la colaboración de otras empresas para que aportaran sus ideas. Todas estas colaboraciones condujeron a la definición de la primera versión de UML.

Desde hace unos años, las tecnologías de la información y comunicación ya han producido una enorme variedad de métodos y notaciones para llevar a cabo el modelado. Existen métodos y anotaciones para el diseño, la estructura, el procesamiento y el almacenamiento de información. De la misma manera también podemos encontrar métodos para la planificación, modelado, implementación, ensamblaje, prueba, documentación, ajuste, etc. de los sistemas. Entre los conceptos que se utilizan existen algunos relativamente fundamentales y, debido a eso, se expanden más allá del ámbito en el que fueron creados en un principio.

Desde la concepción de la tecnología de la información hasta finales de 1970, los desarrolladores de software se tomaron el desarrollo del software como un arte. Pero estos sistemas fueron poco a poco haciéndose más complejos y por esta razón el mantenimiento y el desarrollo exigía otro tipo de visión, más allá del previamente descrito. Este hecho dio lugar a la ya famosa crisis del software, que lleva al enfoque de ingeniería (ingeniería de software) y la programación estructurada. Se desarrollaron métodos para la estructuración de sistemas y para los procesos de diseño, desarrollo y mantenimiento. Los enfoques orientados a procesos, por ejemplo, el método de salida de procesamiento de entrada de jerarquía, enfatizaron la funcionalidad de los sistemas. Con este método, el sistema total se divide en componentes más pequeños a través de la descomposición funcional.

En todos estos métodos y notaciones, dividimos el sistema en dos partes: una sección de datos y una sección de procedimientos. Esto es claramente reconocible en lenguajes de programación más antiguos, como COBOL. Los diagramas de flujo de datos, los diagramas de estructura, los diagramas HIPO y los diagramas de Jackson se utilizan para ilustrar el rango de funciones.

En la década de 1980, el análisis estructural clásico se desarrolló aún más. Los desarrolladores generaron diagramas de relaciones de entidades para el modelado de datos y redes de Petri para el modelado de procesos.

A medida que los sistemas se volvieron más complejos, ya no se podría diseñar cada sistema “desde cero”. Las propiedades, como la mantenibilidad y la reutilización, se hicieron cada vez más importantes. Se desarrollaron lenguajes de programación orientados a objetos, y con ellos, los primeros lenguajes de modelado orientados a objetos surgieron en los años 70 y 80. En la década de 1990, las primeras publicaciones sobre análisis orientado a objetos y diseño orientado a objetos se pusieron a disposición del público. A mediados de la década de 1990, ya existían más de 50 métodos orientados a objetos, así como muchos formatos de diseño. Un lenguaje de modelado unificado parecía indispensable.

A principios de la década de 1990, los métodos orientados a objetos de Grady Booch y James Rumbaugh se utilizaron ampliamente. En octubre de 1994, Rational Software Corporation (parte de IBM desde febrero de 2003) comenzó la creación de un lenguaje de modelado unificado. Primero, acordaron una estandarización de la notación (lenguaje), ya que esto parecía menos elaborado que la estandarización de los métodos. Al hacerlo, integraron el Método Booch de Grady Booch, la Técnica de modelado de objetos (OMT) de James Rumbaugh y la Ingeniería de software orientada a objetos (OOSE), de Ivar Jacobsen, con elementos de otros métodos y publicaron esta nueva notación bajo el nombre UML, versión 0.9.

El objetivo no era formular una notación completamente nueva, sino adaptar, expandir y simplificar los tipos de diagramas existentes y aceptados de varios métodos orientados a objetos, como los diagramas de clase, los diagramas de casos de uso de Jacobson o los diagramas de gráficos de estado de Harel. Los medios de representación que se utilizaron en los métodos estructurados se aplicaron a UML. Por lo tanto, los diagramas de actividad de UML están, por ejemplo, influenciados por la composición de los diagramas de flujo de datos y las redes de Petri.

Modelos con UML

Dentro de la documentación del proyecto, desarrollamos todos los elementos



necesarios para diseñar un plan de trabajo; de modo tal que es momento de continuar con otro documento importante dentro de la gestión de proyectos de desarrollo de software: el/los modelos de arquitectura. Para esta tarea vamos a utilizar un lenguaje de modelado, conocido como UML (Lenguaje Unificado de Modelado).

Pero antes de continuar, analicemos ¿Por qué modelamos?

De la misma forma que justificamos que un arquitecto construya los planos de una casa tantas veces como sea necesario, mostrando diferentes vistas para la obra. El modelado del software es una técnica de ingeniería probada y bien aceptada para construir modelos arquitectónicos de un producto de SW.

¿Qué es un modelo?

Para responder, analicemos la definición de modelo como:

- Un modelo proporciona los planos de cómo es el sistema: muestra una visión global del sistema en consideración, a medida que se avanza tendremos modelos con mayor o menor grado de detalle que van a describir las diferentes perspectivas en cada etapa del ciclo de desarrollo.
- Un modelo es una abstracción de la estructura y comportamiento del sistema, es decir que captura las propiedades estructurales (estática) y de comportamiento (dinámicas) de un sistema.
- Un modelo proporciona plantillas que sirven de guía en la construcción de un programa.

- Un modelo representa y permite visualizar relaciones de trazabilidad entre los diferentes modelos.
- Un modelo documenta las decisiones que se han adoptado.

De este modo, justificada la importancia y relevancia de utilizar un modelo, es momento de preguntarnos:

¿Cómo modelamos?

Unified Modeling Language o UML es un lenguaje estándar para escribir “planos” de software que permiten visualizar, especificar, construir y documentar bloques de información, llamados artefactos, de un sistema que involucra una gran cantidad de software.

Muy importante es aclarar que UML es sólo un lenguaje y NO es una *metodología*; por lo tanto, es sólo una parte de un método de desarrollo de software.

No obstante que UML es independiente del proceso es aconsejable utilizarlo acompañado de un marco de proceso o metodología; como por ejemplo el PUD, conocido como el proceso de desarrollo unificado.

En esencia, UML agrupa notaciones y conceptos provenientes de distintos tipos de métodos orientado a objetos

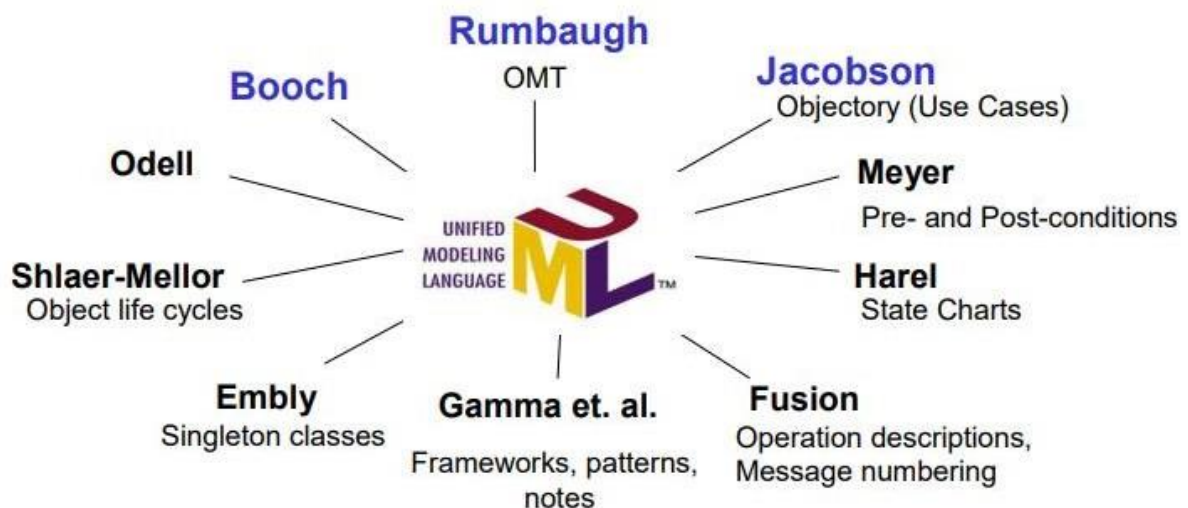


Figura (2) Extraída de internet de UML

Propósito de UML

El modelado proporciona una comprensión de un sistema. Nunca es suficiente con un único modelo, a menudo se necesitan múltiples modelos conectados entre sí. El vocabulario y las reglas de UML indican cómo crear y leer modelos bien formados, pero no dicen qué modelos se deben crear ni cuándo se deberían crear. Así, UML es un lenguaje para:

- **Visualizar:** Un modelo explícito facilita la comunicación. Algunas cosas se modelan mejor textualmente, otras se modelan mejor de forma gráfica. En todos los sistemas interesantes hay estructuras que trascienden lo que pudiese ser representado en un lenguaje de programación. UML es uno de estos lenguajes gráficos, es algo más que un conjunto de símbolos gráficos. Detrás de cada símbolo en la notación UML hay una semántica bien definida. Así, un desarrollador puede construir un modelo en UML y otro desarrollador o incluso otra herramienta, puede interpretar ese modelo sin ambigüedad.
- **Especificar:** En este contexto especificar es construir modelos precisos, no ambiguos y completos. UML cubre la especificación de todas las decisiones de análisis, diseño e implementación que deben realizarse al desarrollar y desplegar un sistema con gran cantidad de software.
- **Construir:** UML no es un lenguaje de programación visual, pero sus modelos pueden conectarse de forma directa a una gran variedad de lenguajes de programación. Esto significa que es posible hacer correspondencias desde un modelo UML a un lenguaje de programación como Java, C++ o Visual Basic, incluso a tablas en una base de datos relacional o al almacenamiento persistente de una base de datos orientada a objetos. Esta correspondencia permite ingeniería directa: la generación de código a partir de un modelo UML en un lenguaje de programación.
- **Documentar:** Una organización de software que trabaje bien produce toda clase de artefactos además de código ejecutable. Estos artefactos incluyen, entre otros: requisitos, arquitectura, diseño, código fuente, planificación de proyectos, pruebas, prototipos, versiones.

Bloques de construcción de UML

Para utilizar UML necesitamos definir tres clases de bloques de construcción: elementos, diagramas y relaciones.

1. Elementos de UML

Existen 4 tipos de elementos en UML.

- a. **Elementos estructurales:** (clases, interfaces, colaboraciones, clases activas, casos de uso, componentes y nodos):

Son los elementos estructurales básicos que se pueden incluir en el modelo UML y son estáticos. También existen variaciones de estos siete elementos, tales como actores, procesos, hilos y aplicaciones, documentos, archivos, bibliotecas, páginas y tablas.

Ejemplo:

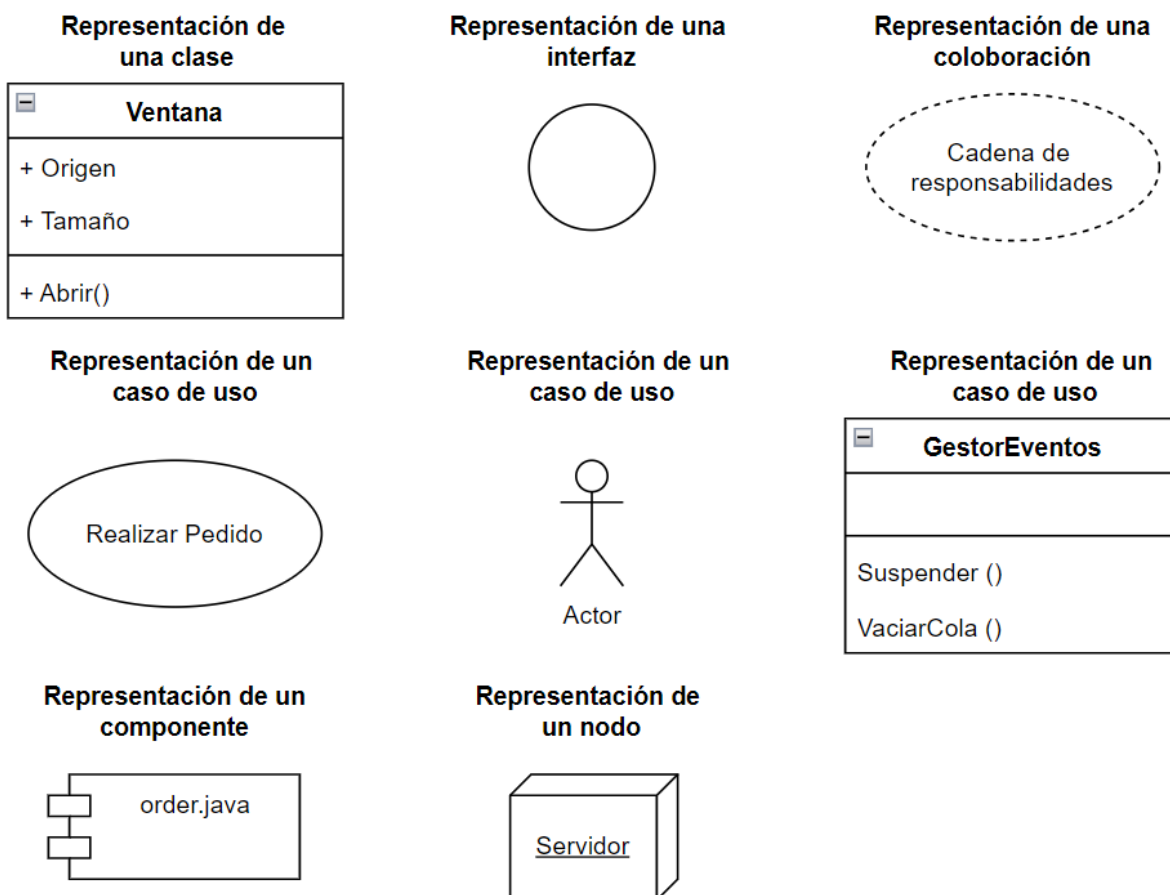


Figura (3) Elaboración propia

b. Los elementos de comportamiento:

Son las partes dinámicas de los modelos UML. Estos son los verbos de un modelo y representan comportamiento en el tiempo y en el espacio. Se clasifican en: interacción y máquina de estados. Estos dos elementos (interacciones y máquinas de estado) son los elementos de comportamiento que se pueden incluir de un modelo UML, estos elementos están conectados normalmente a diversos elementos estructurales, principalmente clases, colaboraciones y objetos.

- **Interacción:** Es un comportamiento que comprende un conjunto de mensajes intercambiados entre un conjunto de objetos, dentro de un contexto particular para alcanzar un propósito específico.
- **Máquina de Estados:** Es un comportamiento que especifica las secuencias de estados por las que pasa un objeto o una interacción durante su vida en respuesta a eventos.

Ejemplo:

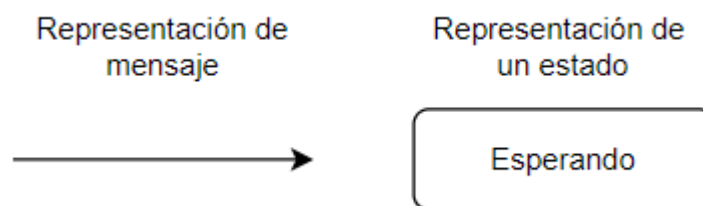


Figura (4) Elaboración propia

c. Los elementos de agrupación:

Son las partes organizativas de los modelos UML. Estos son las cajas en las que pueden descomponerse un modelo. Pueden ser paquetes.

Paquete: es un mecanismo de propósito general para organizar elementos en grupos. En los paquetes se pueden agrupar los elementos estructurales, de comportamiento e incluso otros elementos de agrupación. Los paquetes son los elementos de agrupación básicos con los cuales se puede organizar un modelo UML. También hay variaciones, tales como los frameworks, los modelos y los subsistemas.

Ejemplo

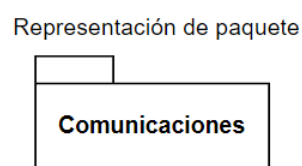


Figura (5) Elaboración propia

d. Los elementos de anotación:

Son las partes explicativas de los modelos UML.

Son comentarios que se añaden para describir, clarificar y hacer observaciones. Hay un tipo principal: Nota: Símbolo para mostrar restricciones y comentarios asociados a un elemento o colección de elementos. Se usan para aquello que se muestra mejor en forma textual (comentario) o formal (restricción).

Ejemplo

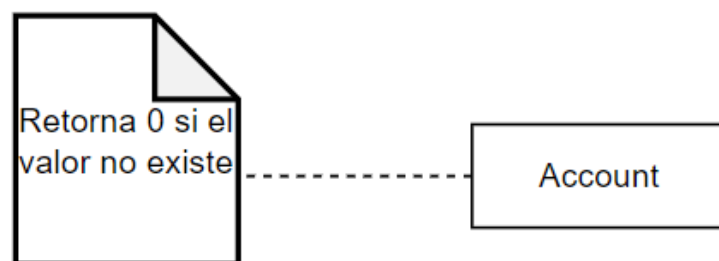


Figura (6) Elaboración propia

2. Relaciones de UML

Una relación define la conexión entre elementos de UML. Para diferenciar las distintas relaciones se utilizan varios tipos de líneas. Nos permiten modelar el enlace entre diferentes elementos estructurales, mostrando además información adicional como multiplicidad (número de instancias de una clase que pueden estar relacionadas con la clase asociada) y nombres de roles (identificación del extremo de una asociación).

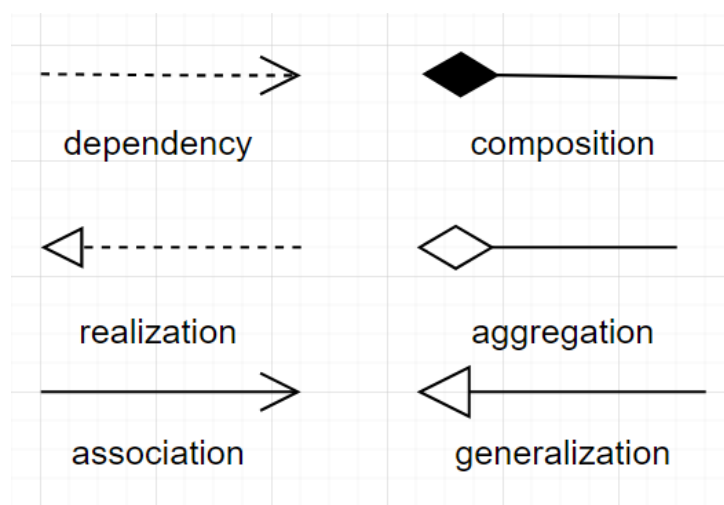


Figura (7) Extraída de internet

- **Asociación:** Es una relación entre dos o más clases, donde una clase utiliza los servicios de la otra. No implica una dependencia fuerte, sino que simplemente existe una conexión entre las clases.

Ejemplo: Un Doctor está asociado con un Paciente. El doctor tiene pacientes a los que atiende, pero ambos pueden existir por separado; es decir, el paciente no depende directamente del doctor para existir.

- **Agregación:** Es un tipo de asociación que indica una relación “todo-parte”. En la agregación, la parte puede existir independientemente del todo. Es decir, si el objeto “todo” se destruye, el objeto “parte” sigue existiendo.

Ejemplo: En un club social, la clase Club podría tener una agregación con la clase Socio. Si el club deja de existir, los socios pueden seguir existiendo como personas independientes.

- **Composición:** Es una relación más fuerte que la agregación. También es una relación “todo-parte”, pero en este caso, las partes no pueden existir independientemente del todo. Si el objeto “todo” se destruye, las partes también lo hacen.

Ejemplo: Un taller mecánico puede tener una relación de composición con Workstation (estaciones de trabajo). Si el taller cierra, las estaciones de trabajo ya no existen por separado, ya que son una parte integral del taller.

- **Dependencia:** Es una relación donde una clase depende de otra para funcionar correctamente, pero esta relación es más débil que la asociación. Indica que un cambio en la clase de la que depende puede afectar a la otra clase

Ejemplo: Un Estudiante depende de un Libro de Texto para aprender sobre un tema específico. Si cambia el libro de texto, el estudiante tendrá que adaptarse al nuevo contenido.

- **Realización:** Es una relación entre una interfaz y una clase concreta que la implementa. La clase se compromete a implementar todos los métodos definidos en la interfaz.

Ejemplo: Un Instructor de Yoga implementa las actividades definidas en un Programa de Yoga. El programa describe los ejercicios y rutinas que se deben seguir, y el instructor lo lleva a cabo.

- **Generalización:** Es una relación de herencia donde una clase hija hereda atributos y comportamientos de una clase padre. La clase hija es una especialización de la clase padre.

Ejemplo: Tanto el Perro y Gato tiene una herencia sobre la clase Animal, heredando comportamientos y atributos comunes, pero implementando los propios.

Más adelante vamos a ver las distintas relaciones a mayor detalle.

3. Diagramas de UML

Los diagramas de UML sirven para visualizar un sistema desde diferentes perspectivas o vistas.

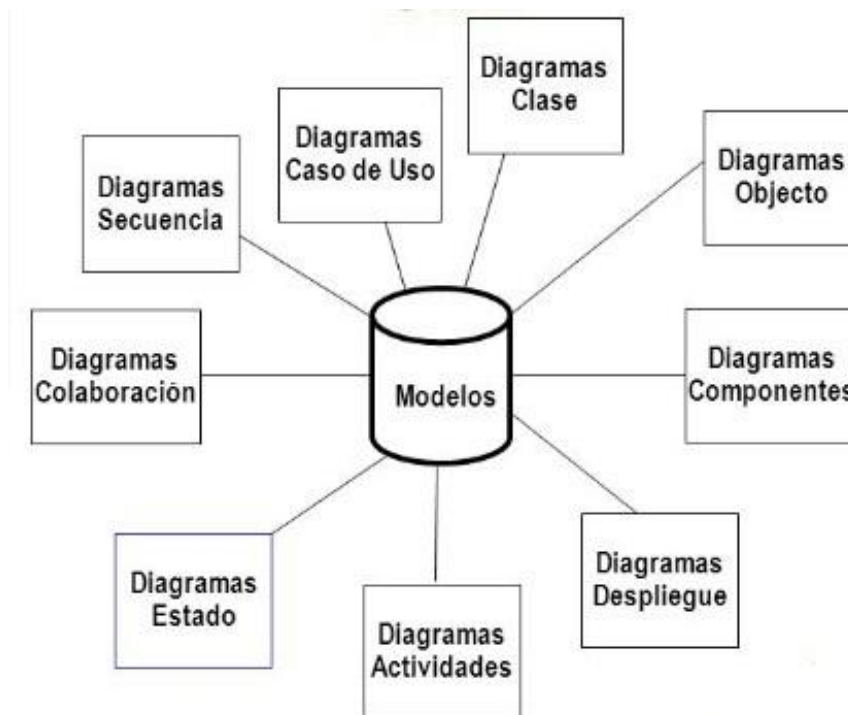


Figura (8) Extraída de internet

El modelo de vistas 4+ 1

El, es un modelo de vistas diseñado por el profesor Philippe Kruchten y que encaja con el estándar “IEEE 1471-2000” (Recommended Practice for Architecture Description of Software-Intensive Systems) que se utiliza para describir la arquitectura de un sistema software intensivo basado en el uso de múltiples puntos de vista.



Este modelo lo que propone Kruchten es que un sistema software se ha de documentar y mostrar (tal y como se propone en el estándar IEEE 1471-2000) con 4 vistas bien diferenciadas y estas 4 vistas se han de relacionar entre sí con una vista más, que es la denominada vista “+1”. Estas 4 vista las denominó Kruchten como: vista lógica, vista de procesos, vista de despliegue y vista física y la vista “+1” que tiene la función de relacionar las 4 vistas citadas, la denominó vista de escenario.

La arquitectura del software se trata de abstracciones, de descomposición y composición, de estilos y estética. También tiene relación con el diseño y la implementación de la estructura de alto nivel del software. Los diseñadores construyen la arquitectura usando varios elementos arquitectónicos elegidos apropiadamente. Estos elementos satisfacen la mayor parte de los requisitos de funcionalidad y performance del sistema, así como también otros requisitos no funcionales tales como confiabilidad, escalabilidad, portabilidad y disponibilidad del sistema.

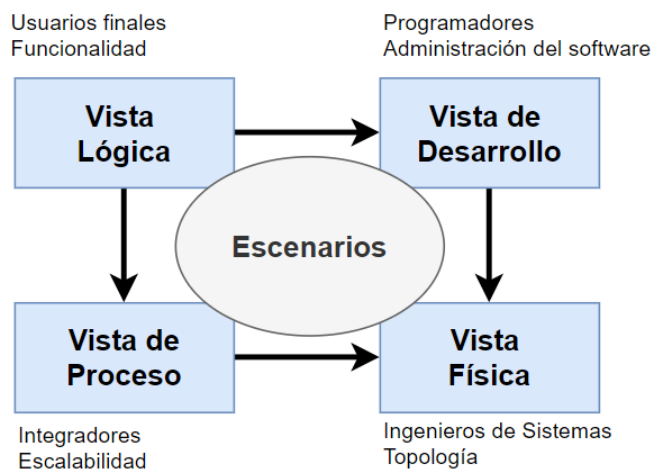


Figura (8) Elaboración propia

- Durante el desarrollo de un sistema software se requiere que éste sea visto desde varias perspectivas.
- Diferentes usuarios miran el sistema de formas diferentes en momentos diferentes.
- La arquitectura del sistema es clave para poder manejar estos puntos de vista diferentes y se organiza mejor a través de vistas arquitecturales interrelacionadas.

Entendemos por arquitectura el conjunto de decisiones significativas sobre:

- La organización de un sistema de software.
- La selección de elementos estructurales y sus interfaces.
- Su comportamiento (colaboraciones entre esos elementos).
- La composición de los elementos estructurales y de comportamiento en subsistemas cada vez más grandes.
- El estilo arquitectónico que guía esta organización (los elementos estáticos y dinámicos y sus interfaces, colaboraciones y su composición).

Las vistas de UML son 5, tal que la vista principal es la vista de casos de uso que permite mostrar los requisitos del sistema, luego la vista de diseño y de procesos que permiten mostrar lo que ocurre dentro del sistema, como la sincronización de procesos y colaboración entre clases. La vista de implementación permite mostrar el cambio de estados de los objetos y modelar aspectos relacionados con requisitos no funcionales y cercano al desarrollo; por ejemplo; representar componentes del sistema.

Finalmente, la vista de despliegue permite mostrar aspectos físicos del sistema como por ejemplo la arquitectura en donde se van a instalar y distribuir los componentes.

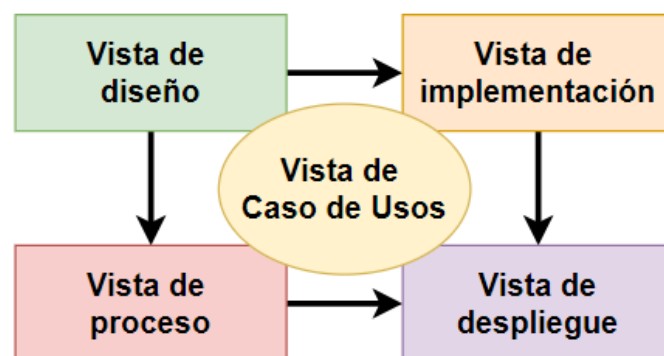


Figura (9) Elaboración Propia

Finalmente, vemos la relación de las vistas y los gráficos que se utilizan en cada una de ellas.

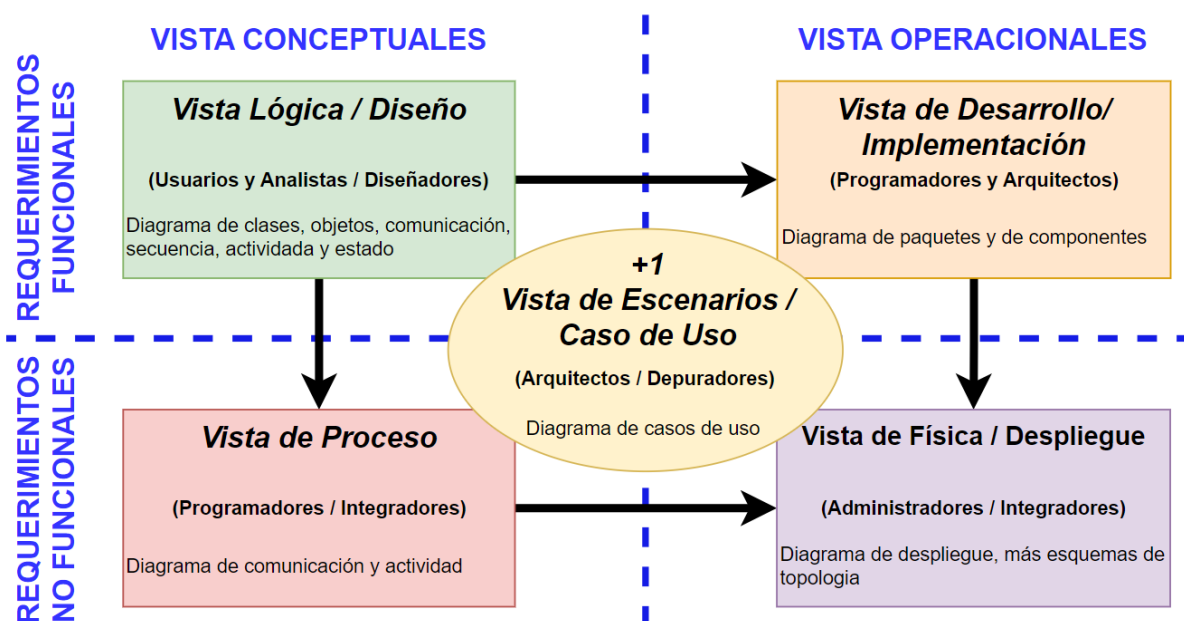


Figura (10) Elaboración Propia

U.M.L y P.U.D

Dentro de los modelos orientados a objetos para desarrollar software de calidad, existe el PUD o Proceso Unificado de desarrollo de software que utiliza UML para conseguir un desarrollo de software con una infraestructura de bloques de construcción del proceso y de contenidos reutilizables.

El PUD tiene las siguientes características principales:

- Está dirigido por los llamados casos de uso: se observa en el grafico que desde los casos de uso se desprende los otros modelos.
- Es centrado en la "arquitectura", como espina dorsal del software.
- Es iterativo e incremental.

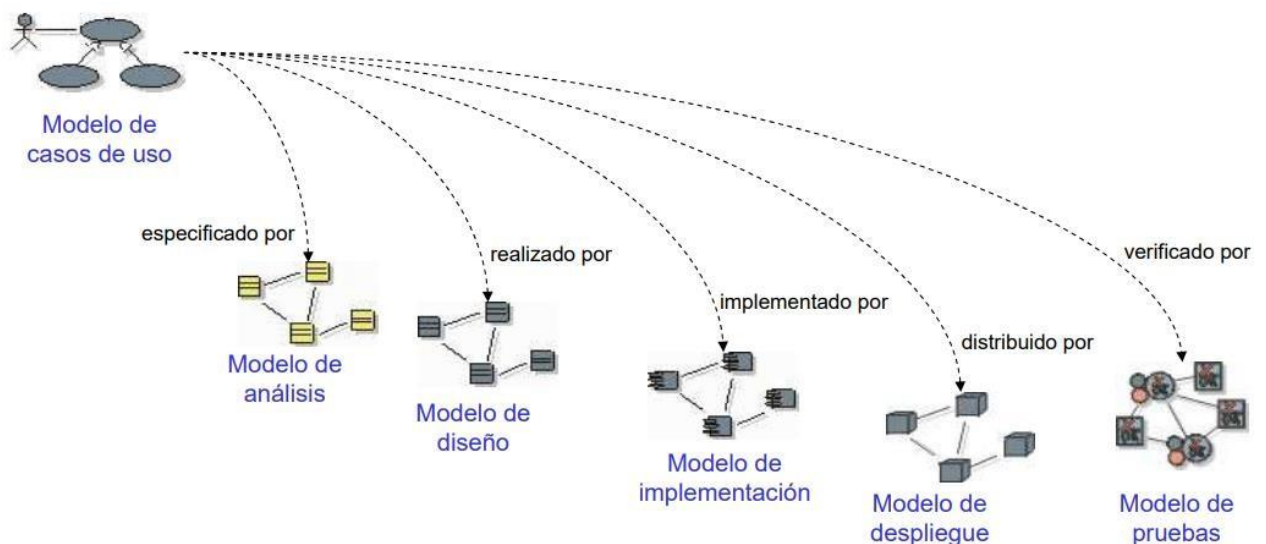


Figura (11) Extraída de internet

El PUD tiene 4 fases y 5 flujos de trabajo:

1. **Fase de inicio**: se desarrolla una descripción del producto final a partir de una idea y se presenta el análisis de negocio para el producto.
2. **Fase de elaboración**: se especifican en detalle la mayoría de los CU y se diseña la arquitectura del sistema.
3. **Fase de construcción**: se crea al producto.
4. **Fase de transición**: se prueba el producto y se informan defectos e deficiencias.

¿Qué es un flujo de trabajo?

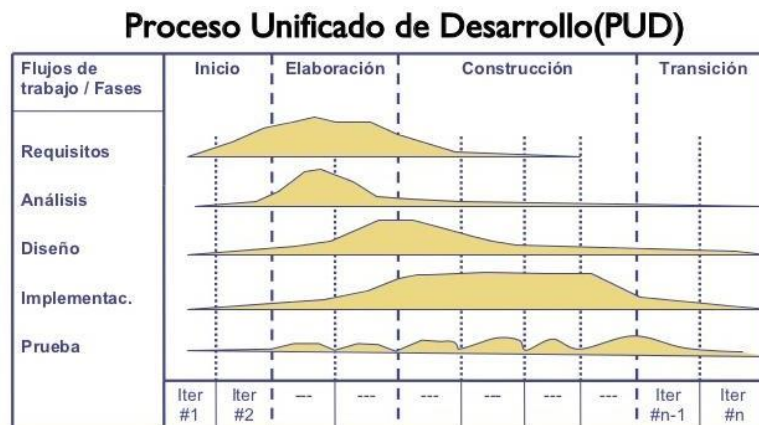


Figura (12) Extraída de internet

Un flujo de trabajo (FT) es un conjunto de actividades, herramientas y estándares que se usan en un proceso. Cada FT está compuesto por: Actividades que son las tareas que se llevan a cabo en el FT; Trabajadores que son quiénes ejecutan las actividades y Artefactos que es toda la información generada a partir del trabajo realizado en el FT.

FT = actividades + trabajadores + artefactos

En el PUD existen cinco flujos de trabajo: FT de Requisitos, FT de Análisis, FT de Diseño, FT de Implementación y FT de Prueba.

Modelo de dominio

El modelo de dominio, también conocido como “Modelo de Objetos del Dominio del Problema” (M.O.D.P) representan las “cosas” que existen o los “eventos” que suceden en el entorno en el que se desenvuelve el sistema.

Muchos de las clases del dominio del problema, pueden deducirse de la especificación de requisitos o mediante la entrevista con los expertos del dominio. Las clases del dominio aparecen como:

Entidades del negocio que representan cosas que se manipulan en el negocio como pedidos, cuentas, contratos, tarifa, reservación. Entidades del mundo real y conceptos de los que el sistema debe hacer un seguimiento, como artículos, vehículos, asiento, avión, pasajero.

Es decir que, en él, se describen las distintas entidades, sus atributos, papeles y relaciones, además de las restricciones que rigen el dominio del problema.

Ejemplo representado mediante un diagrama de clases.

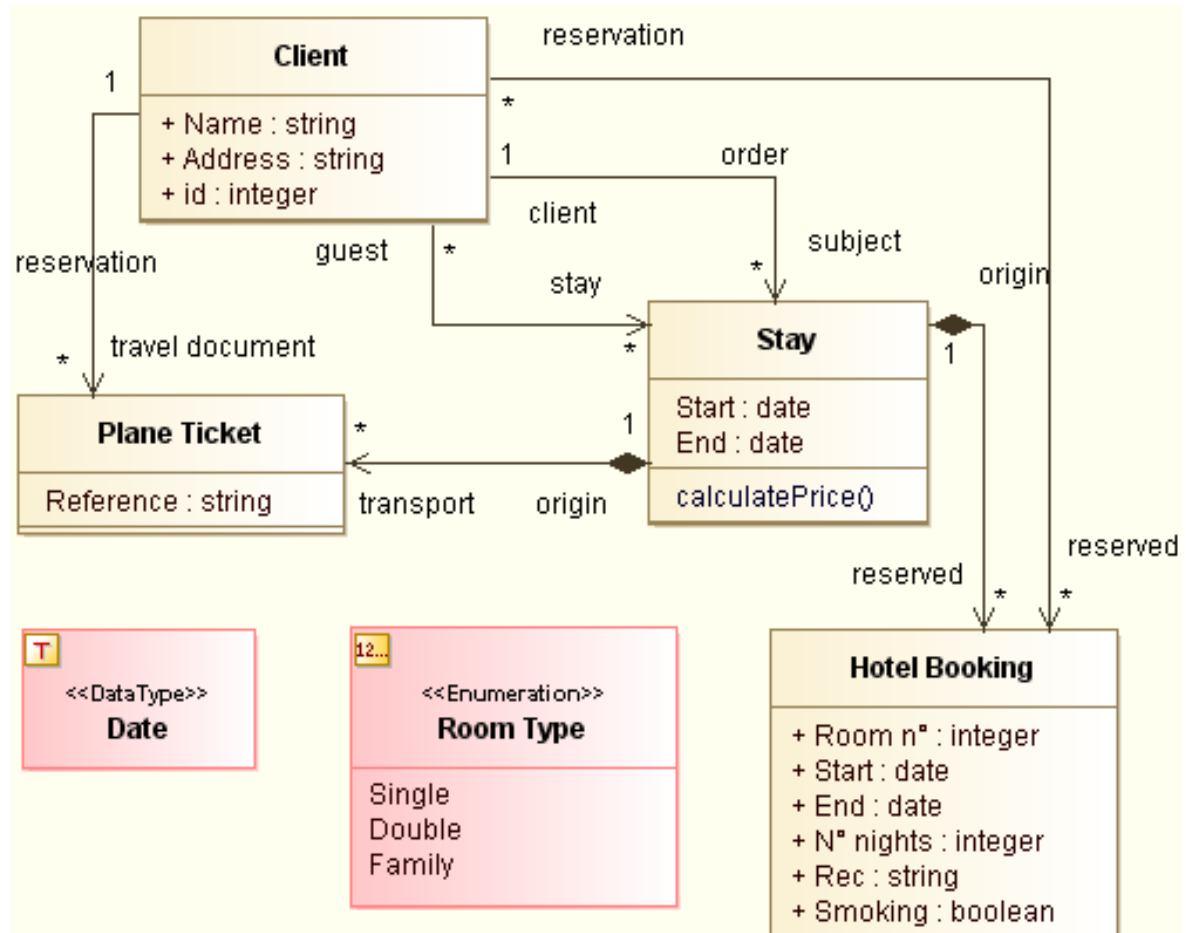


Figura (13) Extraída de internet

El modelado del dominio debe contribuir a la comprensión del problema de modo tal que al observar el modelo se pueda comprender el caso de estudio conocido como el dominio del problema. Este modelo del dominio del problema **NO** es una representación de la base de datos donde el objetivo que se persigue es diferente y se construye en base a reglas diferentes como la normalización.

Para desarrollar este modelo debe conformarse un equipo en el que estén presentes tanto los expertos del dominio (usuarios) como gente con experiencia en modelado, coordinado por los analistas del dominio.

Modelo del dominio: Construcción del Diagrama de Clases

Un concepto esencial en el paradigma orientado a objetos es el hecho de que los objetos “colaboran” o se relacionan entre sí para llevar a cabo un comportamiento superior.

Supongamos que el objeto Caja de Ahorro, que conoce quién es su titular, como parte de su responsabilidad de mostrar sus datos completos debe mostrar el nombre del titular. Entonces necesita pedirle al objeto Titular que le retorne su nombre, para ello le enviará un mensaje indicándole tal solicitud. El objeto Titular responderá retornándole su nombre. Así se manifiesta el enlace entre ambos objetos. Esto puede representarse gráficamente de la siguiente manera:

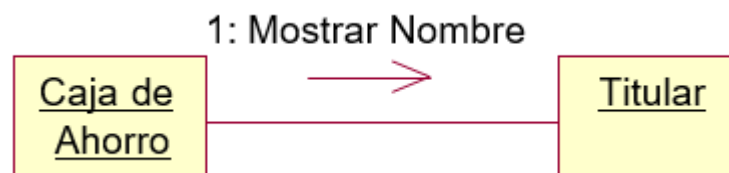


Figura (14) Extraída de internet

Las clases, al igual que los objetos, no existen aisladamente. Antes bien, para un dominio de problema específico las abstracciones clave suelen estar relacionadas por vías muy diversas e interesantes, formando la estructura de clases. El diagrama de clases es un diagrama que muestra dicha estructura mediante un conjunto de clases, interfaces, sus colaboraciones y relaciones.

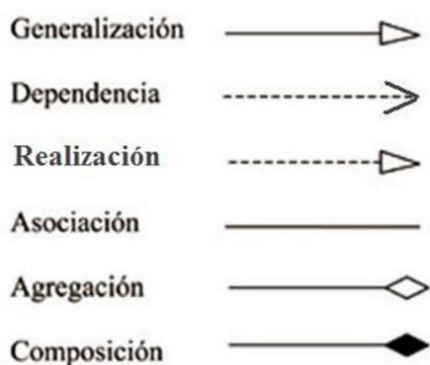
Se puede representar en 2 formas: completo o abreviado.

Con UML los diagramas de clases se emplean para visualizar el aspecto estático de los bloques de construcción básicos del sistema y sus relaciones, y para especificar los detalles para construirlos. En el diagrama de clases podemos encontrar:

- Clases
- Relaciones: herencia, asociación (su variante: agregación), dependencia.
- Indicadores de multiplicidad y navegabilidad
- Nombre de Rol

Relaciones

Cuando diferentes objetos trabajan en función a una meta o propósito; dicha meta se logra por medio de la colaboración entre las partes a través de las relaciones. Por tanto, las relaciones son la conexión entre elementos de UML que pueden ser clases, componentes, paquetes y casos de uso.



De los diferentes tipos de relaciones vamos a analizar: la asociación como la más básica, y avanzaremos sobre el estudio de relaciones más complejas como la dependencia y la generalización. También existe la relación de realización, pero no es objeto de nuestro estudio. Recordemos la representación gráfica que provee de UML para cada tipo de relación.

Ejemplo de asociación

Las asociaciones representan las relaciones más generales entre clases, es decir, las relaciones con menor contenido semántico. La forma de representar las asociaciones binarias en UML es mediante una línea que conecta las dos clases. Gráficamente, una asociación se representa como una línea continua que conecta la misma o diferentes clases:

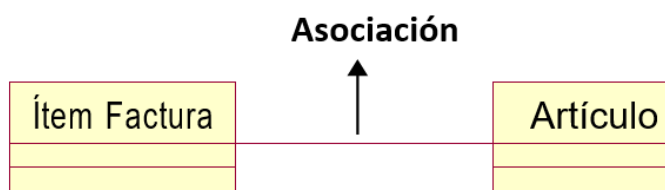


Figura (17) Extraída de internet

Navegación

Un concepto importante para modelar respecto de una asociación es la **navegación**. La navegación de una asociación especifica que dado un objeto perteneciente a la clase de un extremo se puede llegar fácil y directamente a los objetos de la clase del otro extremo, normalmente debido a que el objeto inicial almacena algunas referencias a los objetos en el destino. La dirección de la **navegación** indica qué clase es la que contiene la referencia hacia la otra (determina “quién conoce a quién”).

En este ejemplo Factura conoce a Cliente

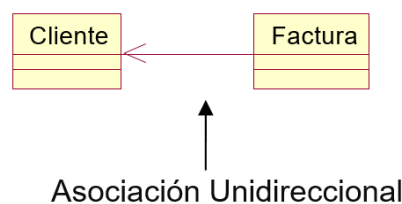


Figura (18) Elaboración propia

Puede ocurrir que en algunos casos la asociación necesite ser navegable en ambos sentidos. Por ejemplo:

Para mostrar los datos completos de una factura es necesario que factura conozca al cliente al que corresponde.

Para mostrar un resumen de cuenta de un cliente es necesario que cliente conozca las facturas que le corresponden.

Esta situación se modela de la siguiente manera:

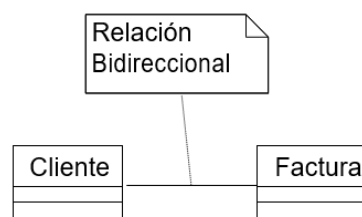


Figura (19) Elaboración propia

O también se puede representar así:



Figura (20) Elaboración propia

En general, las asociaciones son bidireccionales, esto es, no tienen un sentido asociado.

Multiplicidad

En muchas situaciones de modelado, es importante señalar cuántos objetos pueden conectarse a través de una instancia de la asociación. Este “cuántos” se denomina **multiplicidad** de la asociación y se escribe como una expresión que se evalúa a un rango de valores o a un valor explícito.

Cuando se indica una multiplicidad en un extremo de una asociación se está especificando que para cada objeto de la clase en el extremo opuesto debe haber tantos objetos en este extremo. Se puede indicar una multiplicidad de exactamente uno (1), cero (0), muchos (*), uno o más (1..*) o un valor exacto (por ejemplo, 3). La multiplicidad se indica en el extremo que corresponde a la navegación.

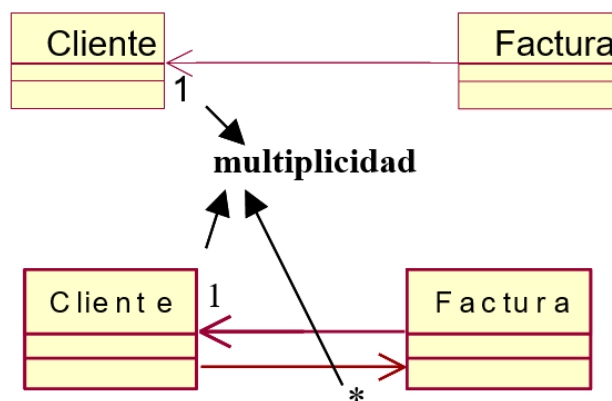


Figura (21) Elaboración propia

Ejemplo de Agregación:

- Es una Relación dinámica: el tiempo de vida del objeto que se agrega es independiente del objeto agregador.
- El objeto agregador utiliza al agregado para su funcionamiento.
- SIMILAR parámetro pasado “por referencia”.

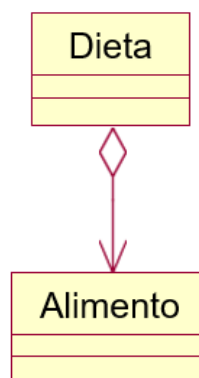


Figura (22) Elaboración propia

Una determinada dieta puede dejar de existir, pero ello no implica que ocurra lo mismo con los alimentos que ella contenía. Estos pueden seguir existiendo y estar contenidos en otras dietas

Ejemplo de Composición

- Es una relación estática: el tiempo de vida del objeto incluido está condicionado por el tiempo de vida del objeto compuesto.
- El objeto compuesto se construye a partir del objeto incluido.
- SIMILAR parámetro pasado “por valor”. La composición implica que los componentes de un objeto sólo pueden pertenecer a un solo objeto agregado, de forma que cuando el objeto agregado es destruido todas sus partes son destruidas también.

Un determinado ítem no puede dejar de existir ya que no puede seguir existiendo y estar contenidos en otra factura. (El diamante va pintado en color)

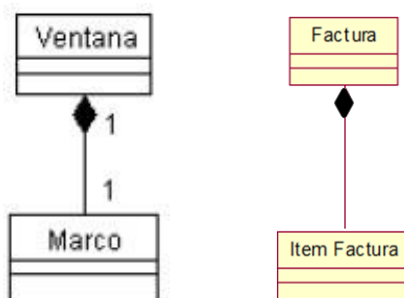


Figura (23) Elaboración propia

Ejemplo de Generalización

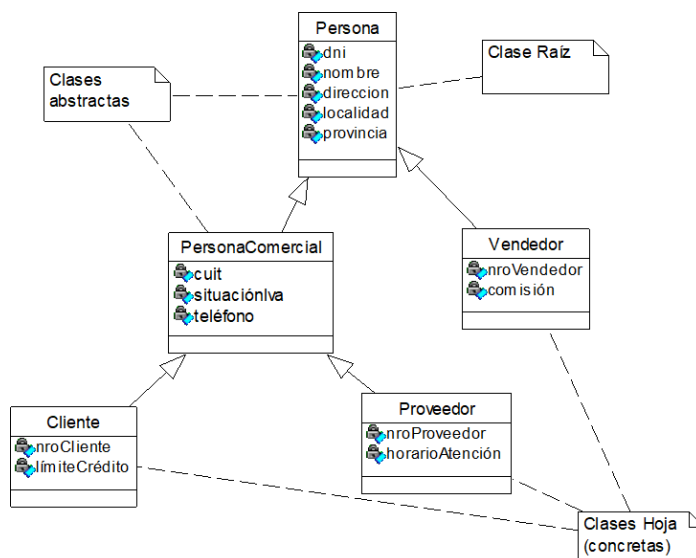


Figura (24) Elaboración propia

La generalización es la típica relación de herencia/especialización entre clases. La herencia es una implantación de la generalización. La generalización establece que las propiedades de un tipo se aplican a sus subtipos. La herencia de estructura de datos permite la reutilización de la estructura.

En UML la herencia se representa mediante una flecha, cuya punta es un triángulo vacío. La flecha que representa a la generalización va orientada desde la subclase a la superclase.

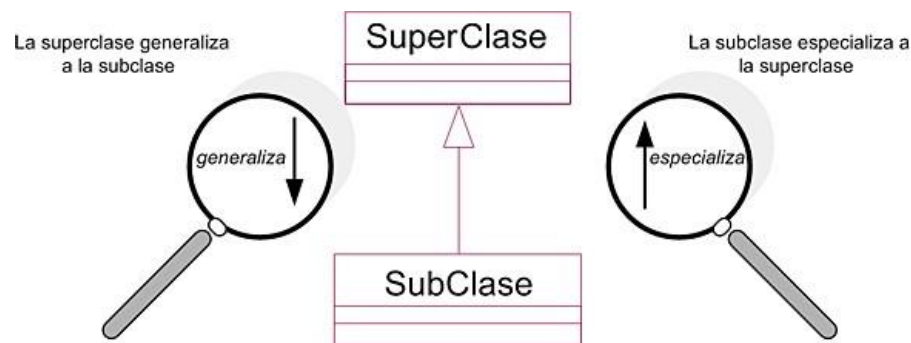


Figura (25) Elaboración propia

Cuando de una superclase se derivan varias subclases existen dos notaciones diferentes, aunque totalmente equivalentes, para su representación. Una generalización da lugar al polimorfismo entre clases de una jerarquía de generalizaciones: Un objeto de una subclase puede sustituir a un objeto de la superclase en cualquier contexto. Lo inverso no es cierto.

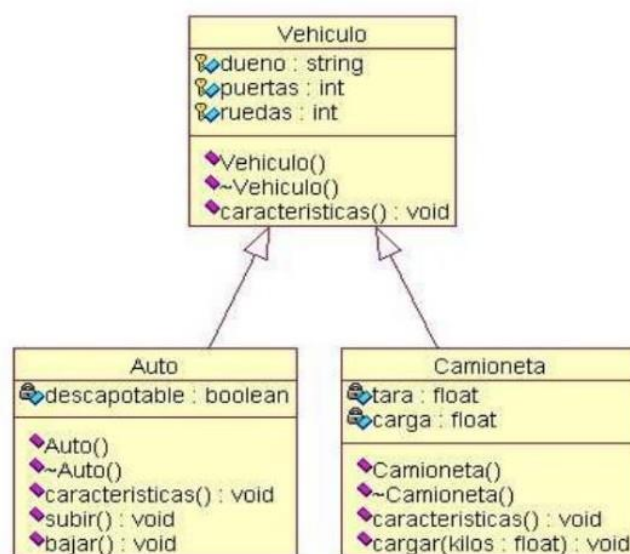


Figura (26) Elaboración propia

Ejemplo: **Un auto “es un” vehículo. Una camioneta “es un” vehículo.**

Ejemplo de Dependencia

Gráficamente estas relaciones se representan con una línea discontinua dirigida hacia la clase de la cual se depende, como se muestra a continuación:

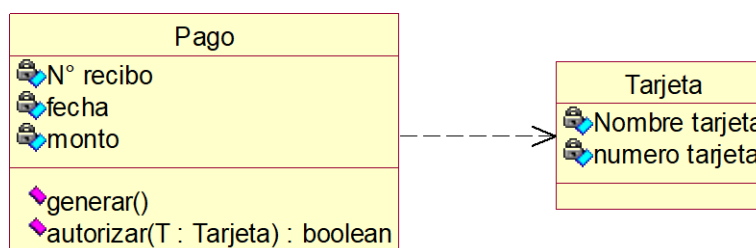


Figura (27) Elaboración propia

Este tipo de relación se define como “una relación de uso que declara que un cambio en la especificación de un elemento puede afectar a otro elemento que la utiliza”, esto representa la dependencia que tiene una clase de otra.

Generalmente las dependencias se utilizan para indicar que una clase utiliza a otra como argumento en la signatura de una operación. Esto es claramente una relación de uso: si la clase utilizada cambia, la operación de la otra clase puede verse también afectada, porque la clase utilizada puede presentar ahora una interfaz o comportamientos diferentes.

La dependencia se utiliza para representar la visibilidad de parámetro. En nuestro ejemplo Tarjeta se hace visible para Pago porque un objeto de esta última clase recibirá como parámetro de entrada para su operación “autorizar”, un objeto de la clase Tarjeta.

Modelo del dominio: Construcción del Diagrama de Casos de uso

Los casos de uso proporcionan un medio intuitivo y sistemático para capturar los requisitos funcionales. La utilización de los casos de uso hace que los analistas deben pensar en términos de quiénes son los usuarios y qué necesidad de funcionalidades requieren cada actor.

El principal esfuerzo de la fase de requisitos es desarrollar un modelo del sistema que se va a construir y la utilización de los casos de uso es una forma adecuada para ello, en UML se deben respetar, reglas para graficar casos de uso. Por ejemplo:

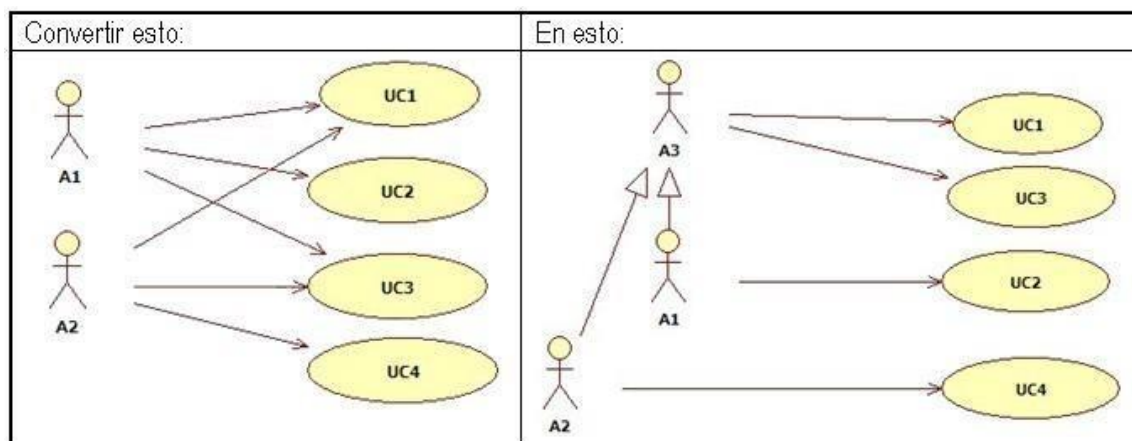


Figura (28) Elaboración propia

El modelo de casos de uso permite que los desarrolladores y los clientes lleguen a un acuerdo sobre los requisitos, es decir sobre lo que debe cumplir el sistema y constituye la entrada principal para el análisis, el diseño y las pruebas.

Los diagramas de casos de uso son importantes para modelar el comportamiento de un sistema o un subsistema. Estos diagramas facilitan que los sistemas y subsistemas sean abordables y comprensibles, al presentar una vista externa de cómo pueden utilizarse estos elementos en un contexto dado.

Los diagramas de casos de uso contienen: casos de uso, actores, relaciones de dependencia, generalización y asociación, tal que los actores se conectan a los casos de uso a través de asociaciones.

Una asociación entre un actor y un caso de uso indica que el actor y el caso de uso se comunican entre sí y cada uno puede enviar y recibir mensajes.

Simbología del Diagrama de Casos de Uso



Figura (29) Elaboración propia

- **Actor**

Un actor es el rol que juega un usuario en un caso de uso. Normalmente, un actor representa un rol que es jugado por una persona, un dispositivo de hardware o incluso otro sistema al interactuar con nuestro sistema, es un usuario del sistema. El modelo de casos de uso describe lo



que hace el sistema para cada tipo de usuario. Cada usuario puede representarse por uno o más actores. De la misma forma cada sistema externo que interactúa con el sistema en desarrollo puede representarse por uno o más actores.

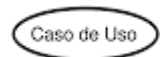
Cada vez que un usuario en concreto (un humano u otro sistema) interactúa con el sistema, la instancia correspondiente del actor está desarrollando ese papel. Una instancia de un actor es, por lo tanto, un usuario concreto que interactúa con el sistema.

Nombre de actor

El nombre del actor debe reflejar el rol que cumple un usuario al interactuar con el caso de uso al que está conectado.

- **Caso de Uso**

Un caso de uso representa cada forma en que los actores usan el sistema. Los casos de uso son fragmentos de funcionalidad que el sistema ofrece para aportar un resultado de valor para sus actores.



Nombre de caso de uso

Cada caso de uso debe tener un NUMERO y un NOMBRE que lo distinga de los demás. Puede ser un nombre simple (una simple cadena de texto) o un nombre de camino en el caso de que esté precedido por el nombre del paquete en que está incluido el caso de uso. Los nombres de casos de uso son expresiones verbales que describen algún comportamiento del vocabulario del sistema que se está modelando.

Nro + Verbo en infinitivo + (objeto del dominio del problema)

Un caso de uso especifica una secuencia de acciones que el sistema puede llevar a cabo interactuando con sus actores, incluyendo secuencias dentro de la secuencia.

Relación	Símbolo	Significado
Comunica	—————	Para conectar un actor con un caso de uso se utiliza una línea sin puntas de flecha.
Incluye	<< Incluye >> ←-----	Un caso de uso contiene un comportamiento común para más de un caso de uso. La flecha apunta al caso de uso común.
Extiende	<< Extiende >> ----->	Un caso de uso distinto maneja las excepciones del caso de uso básico. La flecha apunta del caso de uso extendido al básico.
Generaliza	—————>	Una "cosa" de UML es más general que otra "cosa". La flecha apunta a la "cosa" general.

Figura (30) Elaboración propia

Relación de generalización/inclusión y extensión entre casos de uso

Los casos de uso pueden organizarse especificando relaciones de generalización, inclusión y extensión entre ellos. Estas relaciones se utilizan para factorizar el comportamiento común (extrayendo ese comportamiento de los casos de uso en los que se incluye) y para factorizar variantes (poniendo ese comportamiento en otros casos de uso que lo extienden).

- **Generalización**

La generalización entre casos de uso es como la generalización entre clases. En este contexto significa que el caso de uso hijo hereda el comportamiento del caso de uso padre. El hijo puede modificar y/o agregar comportamiento al heredado.

La generalización se emplea para simplificar la forma de trabajo y la comprensión del modelo de casos de uso y para reutilizar casos de uso "semifabricados". Un caso de uso "semifabricado" existe solamente para que otros lo reutilicen.

Gráficamente se indica, al igual que la herencia entre clases, con una línea continua con punta de flecha vacía dirigida del caso de uso hijo hacia el padre.

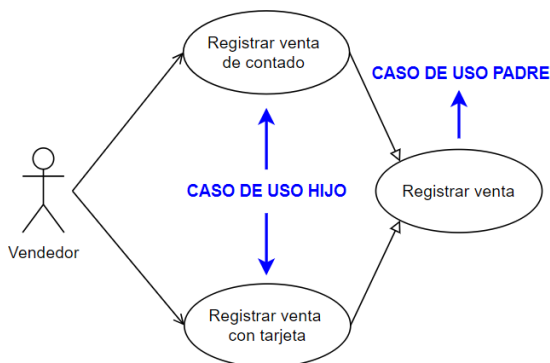


Figura (31) Elaboración propia

- **Inclusión**

Una relación de inclusión entre casos de uso significa que un caso de uso base incorpora explícitamente el comportamiento de otro caso de uso en el lugar especificado en el caso base. El caso de uso incluido nunca es instanciado por un actor, sino que es instanciado por el/los casos de uso que lo incluyen. Por ello, un caso de uso de inclusión es siempre un caso de uso abstracto.

La relación de inclusión se usa para abstraer el comportamiento común entre casos de uso, evitando describir el mismo flujo de eventos repetidas veces. La secuencia de comportamiento y los atributos del caso de uso incluido se encapsulan y no pueden modificarse o accederse, solamente puede utilizarse el resultado (o función) del caso de uso incluido.

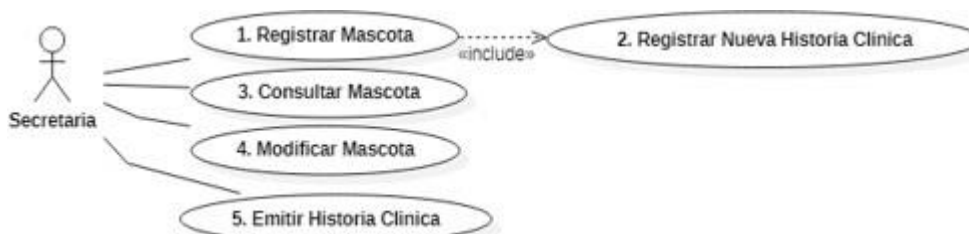


Figura (32) Elaboración propia

La inclusión se representa como una dependencia estereotipada con la palabra <<include>> o en forma abreviada <<inc>>, dirigida desde el caso de uso base hacia el caso de uso incluido.

- **Extensión**

Una relación de extensión entre casos de uso significa que un caso de uso base incorpora implícitamente el comportamiento de otro caso de uso. El caso de uso base puede ejecutarse aisladamente, pero bajo ciertas condiciones su funcionalidad será extendida por la del caso de uso de extensión.

La extensión puede verse como que el caso de uso que extiende incorpora su comportamiento en el caso de uso base. Una relación de extensión se utiliza para modelar la parte de un caso de uso que el usuario puede ver como comportamiento opcional del sistema. De esta forma se separa el comportamiento opcional del obligatorio. También puede utilizarse para modelar un subflujo separado que sólo se ejecuta bajo ciertas circunstancias.

El caso de uso de extensión puede en algunos casos ser instanciado directamente por un actor (en este caso se considera un caso de uso *concreto*), además de instanciarse para extender el comportamiento de un caso de uso base. Si el caso de uso de extensión nunca es instanciado directamente por un actor, entonces es un caso de uso *abstracto*.

La extensión se representa como una dependencia estereotipada con la palabra *extend* o en forma abreviada *ext*, dirigida desde el caso de uso de extensión hacia el caso de uso base.

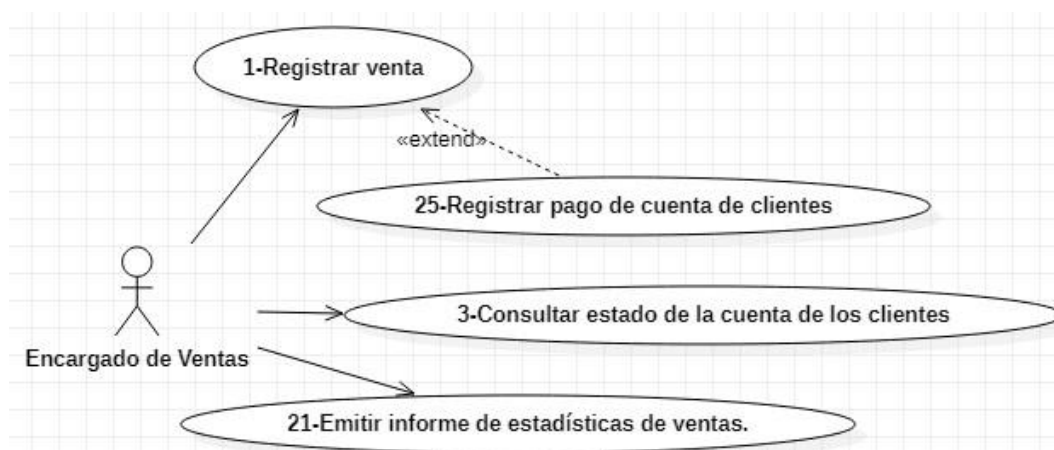


Figura (33) Elaboración propia

Si el modelo de casos de uso es grande, es decir si el número de ellos es elevado es útil introducir el concepto de “Paquete” y representarlo con UML en el modelo para tratar su tamaño.

Un paquete reunirá cierto número de casos de uso agrupados por algún criterio de homogeneidad: los que corresponden a un mismo actor, los que se refieren a un mismo proceso, etc.

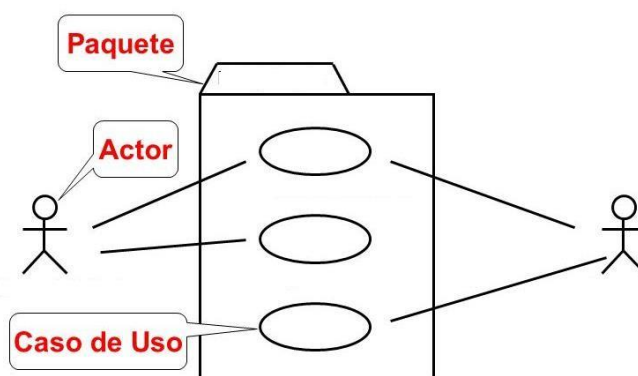


Figura (34) Elaboración propia

Flujo de análisis

Diagrama de interacción

La secuencia de acciones en un **CASO DE USO** comienza cuando un actor invoca el caso de uso mediante el envío de un mensaje al sistema. Entonces, un objeto de interfaz recibirá ese mensaje, a su vez, enviará un mensaje a algún otro objeto y de esta forma los objetos interactúan y se comunican para llevar a cabo el caso de uso.

Hay dos tipos de diagramas de interacción: el diagrama de colaboración y el diagrama de secuencia. En análisis es preferible utilizar el diagrama de colaboración, ya que aquí el objetivo fundamental es identificar responsabilidades de los objetos y no tiene tanta importancia la secuencia detallada y ordenada cronológicamente. Por lo tanto, dejaremos los diagramas de secuencia para la actividad de diseño.

Diagrama de colaboración

Este diagrama muestra como los objetos y los actores intercambian mensajes y colaboran entre sí para cumplir el objetivo de un caso de uso. En este diagrama se utilizan tres tipos de clases: de entidad, de interfaz y de control.

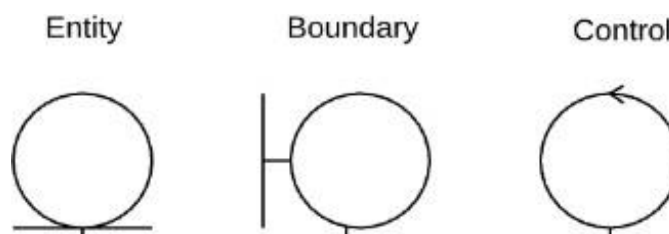


Figura (36) Elaboración propia

Este diagrama es el modelo lógico para representar el concepto del Modelo vista controlador. (MVC)

Veamos un ejemplo del diagrama.

Una interacción es un

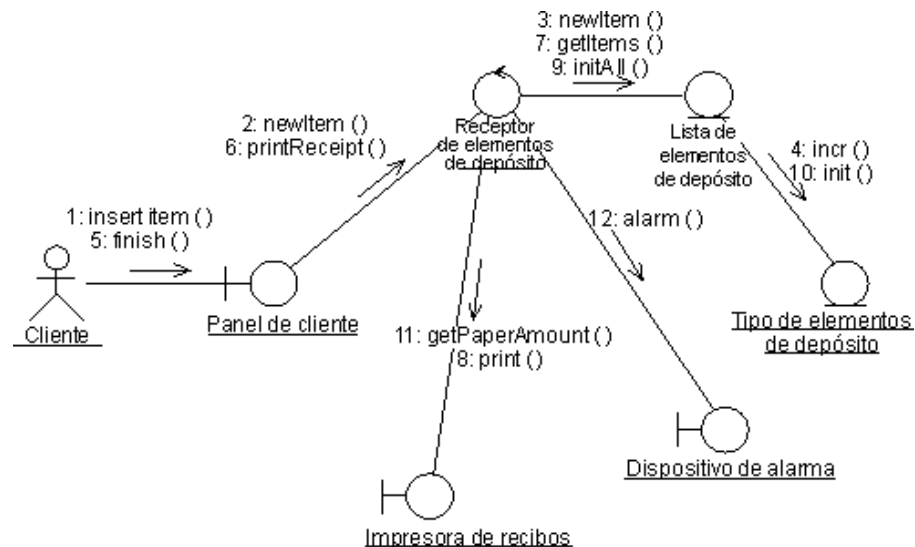


Figura (37) Extraída de Internet UML

Comportamiento que comprende un conjunto de mensajes intercambiados entre un conjunto de objetos dentro de un contexto para lograr un propósito. El actor que hace peticiones al sistema a través del panel del cliente.

Un **mensaje** es la especificación de una comunicación entre objetos que transmite información, a la espera de que se desencadene una actividad.

Un **enlace** es una conexión semántica entre dos objetos. Es una instancia de una asociación.

También, pueden ser representados con otros estereotipos.

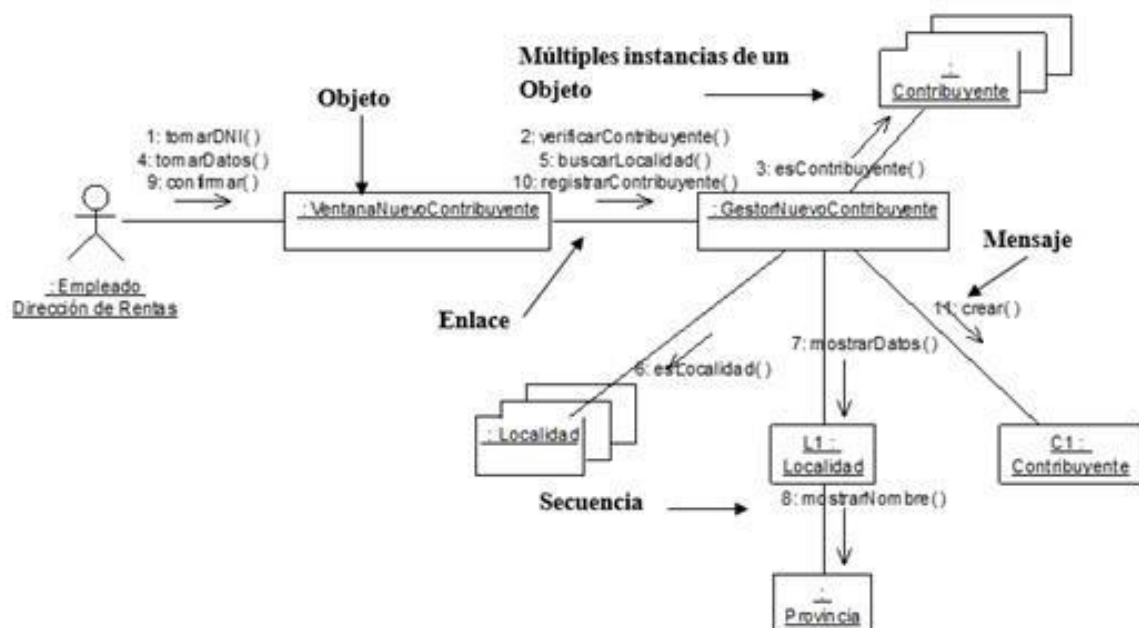


Figura (38) Extraída de Internet UML

Flujo de diseño

En este flujo se puede utilizar un diagrama de estado o un diagrama de secuencia. Veamos ambos diagramas.

Diagrama de secuencia

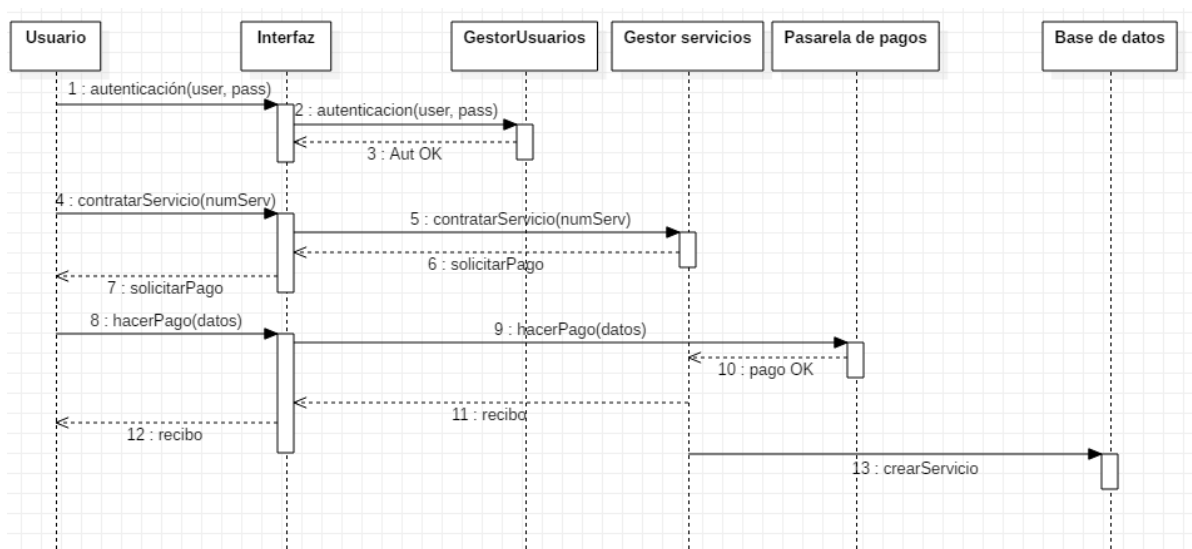


Figura (39) Extraída de Internet UML

Este diagrama muestra la colaboración y secuencia de mensajes entre las clases de entidad, interfaz y controlador a partir de las peticiones del actor principal del caso de uso. Se suele construir para comprender mejor el diagrama de clases, ya que el diagrama de secuencia muestra como objetos de esas clases interactúan haciendo intercambio de mensajes. El diagrama de secuencia está construido a partir de dos dimensiones: Horizontal: Representa los objetos que participan en la secuencia. Vertical: Representa la línea de tiempo sobre la que los elementos actúan

Modelo de diseño: Construcción del Diagrama de Estado

Los diagramas de estado muestran el conjunto de estados por los cuales pasa un objeto durante su vida en una aplicación en respuesta a eventos (por ejemplo, mensajes recibidos, tiempo rebasado o errores), junto con sus respuestas y acciones. También ilustran qué eventos pueden cambiar el estado de los objetos de la clase. Como los estados y las transiciones incluyen, a su vez, eventos, acciones y actividades, vamos a ver primero sus definiciones.

Se representan con un estado, evento, estado inicial y estado final.

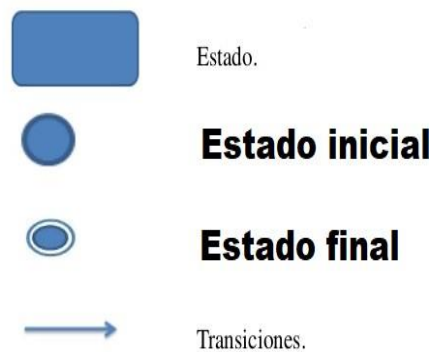


Figura (40) Extraída de Internet UML

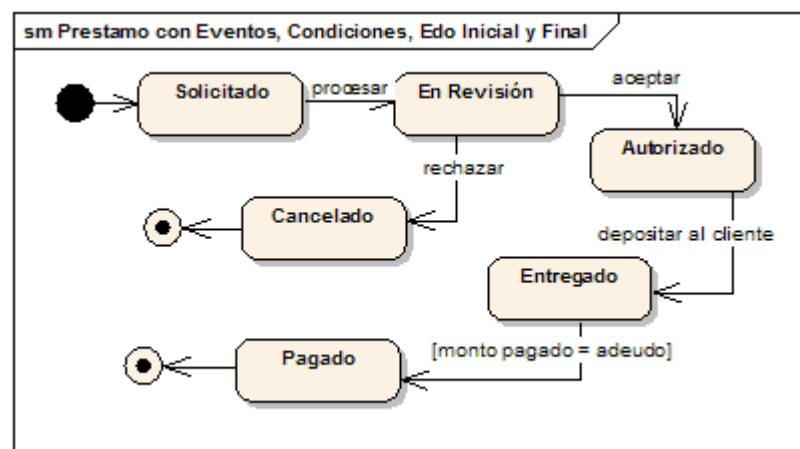


Figura (41) Extraída de Internet UML

Este diagrama puede ser implementado mediante estados anidados y estados compuestos.

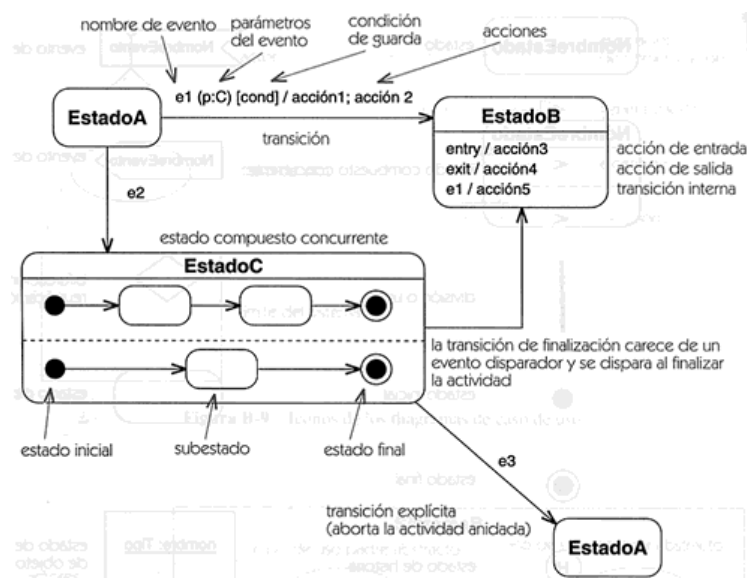


Figura (42) Extraída de Internet UML

Diagrama de actividad

El diagrama de actividad es un diagrama UML de comportamiento que muestra el flujo de control o el flujo de objetos, con especial énfasis en la secuencia y las condiciones de este flujo.

Estos diagramas son utilizados para describir cualquier tipo de procesos, es especialmente común para modelar de forma gráfica los diferentes casos de uso, transacciones o procedimientos que haya en un sistema de información. En resumen, son utilizados para representar la forma en la que un sistema hace una implementación.

Los diagramas de actividades muestran una secuencia de acciones, un flujo de trabajo que va desde un punto inicial hasta un punto final. Vemos un ejemplo para la clase: MESA

ad Activity (Example)

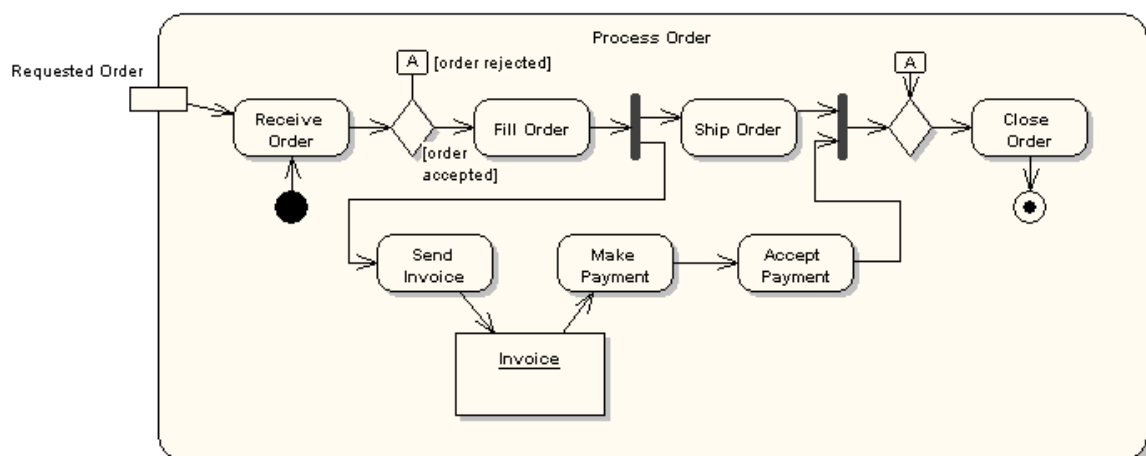


Figura (43) Extraída de Internet UML

Flujo de implementación

Aquí se utilizan los diagramas de componentes y despliegue.

Diagrama de componentes

El diagrama de componentes proporciona una vista física de la construcción del sistema de información. Muestra la organización de los componentes software, sus interfaces y las dependencias entre ellos.

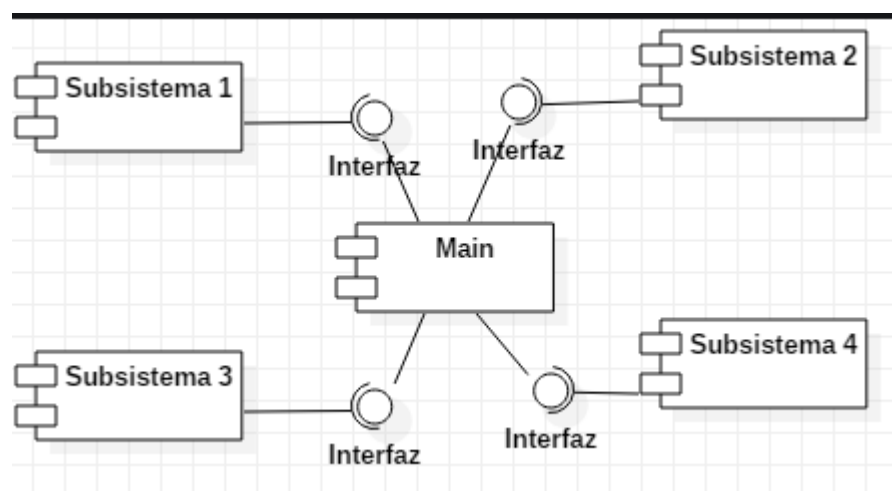


Figura (44) Extraída de Internet UML

Componente

Un componente se representa como un rectángulo, con dos pequeños rectángulos superpuestos perpendicularmente en el lado izquierdo.

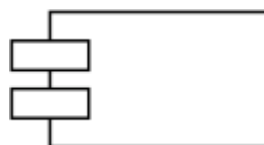


Figura (44) Extraída de Internet UML

Para distinguir distintos tipos de componentes se les puede asignar un estereotipo, cuyo nombre estará dentro del símbolo: << ... >>

Interfaz

Se representa como un pequeño círculo situado junto al componente que lo implementa y unido a él por una línea continua. La interfaz puede tener un nombre que se escribe junto al círculo. Un componente puede proporcionar más de una interfaz.



Figura (45) Extraída de Internet UML

Paquete

Un paquete se representa con un icono de carpeta. Veamos un ejemplo

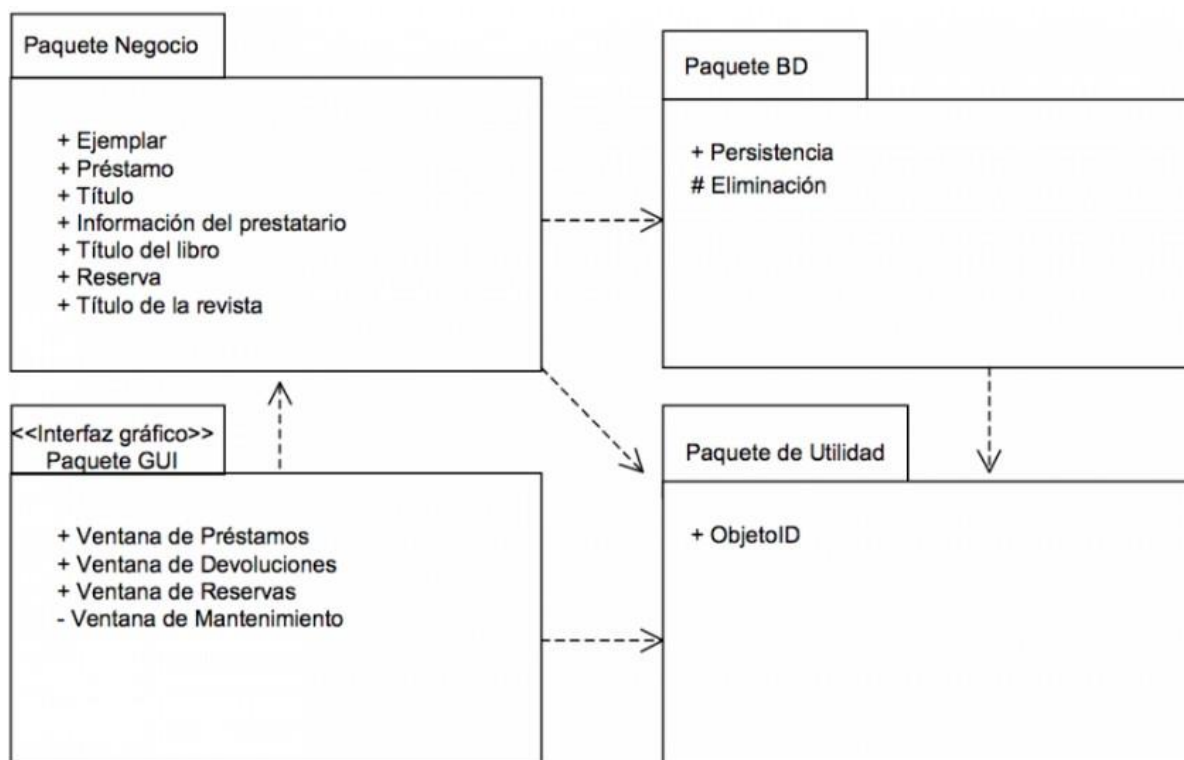


Figura (46) Extraída de Internet UML

Relación de dependencia

Una relación de dependencia se representa mediante una línea discontinua con una flecha que apunta al componente o interfaz que provee del servicio o facilidad al otro. La relación puede tener un estereotipo que se coloca junto a la línea, entre el símbolo: <<...>>

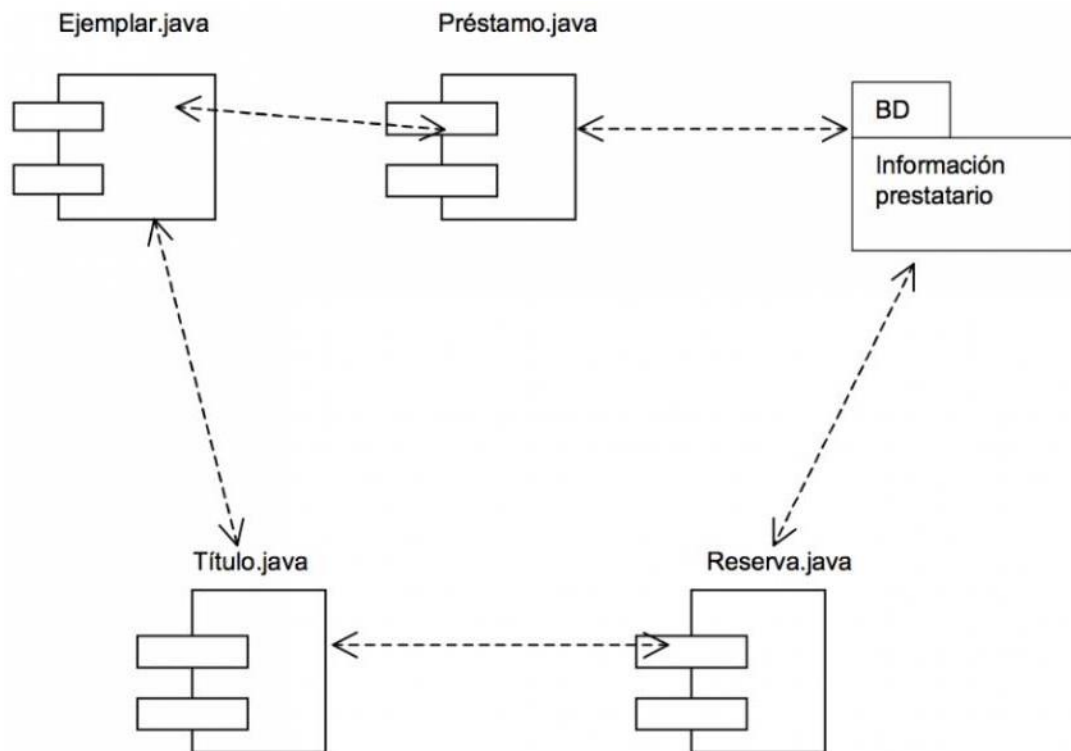


Figura (47) Extraída de Internet UML

Diagrama de despliegue.

El diagrama de despliegue es otro de los diagramas de estructura del conjunto de los diagramas de UML 2.5. Es utilizado para representar la distribución física (estática) de los componentes software en los distintos nodos físicos de la red. Suele ser utilizado junto con el diagrama de componentes (incluso a veces con el diagrama de paquetes) de forma que, juntos, dan una visión general de cómo estará desplegado el sistema de información.

El diagrama de componentes utiliza, principalmente, dos tipos de elementos: Nodos y conexiones.

Nodos

Los nodos se definen como elementos utilizados para representar un elemento físico que interactúa de alguna manera con el sistema o bien forma parte de este. Se representa utilizando un cubo tridimensional:

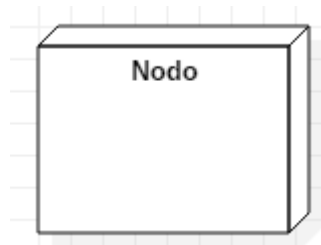


Figura (48) Extraída de Internet UML

Notación de un nodo

Algunos ejemplos de nodos podrían ser: Servidor web, Servidor DNS, Servidor de Aplicaciones, PC Usuario, Base de datos, etc.

Los nodos también pueden ser representados utilizando **iconos personalizados** con la finalidad de clarificar el contenido del diagrama. Algunos de estos iconos de uso extendido son:

- Un muro para representar un Firewall.
- Un icono de un PC para representar el equipo de un usuario.
- Un círculo con flechas para identificar a un router.
- Una nube para representar una WAN (aunque no es propiamente un nodo)
- Un cilindro para representar una base de datos.

Un nodo a su vez puede tener nodos incluidos en su interior, dando a conocer que son sistemas separados incluidos dentro del mismo nodo físico. De esta forma se compondrían los **nodos compuestos**.

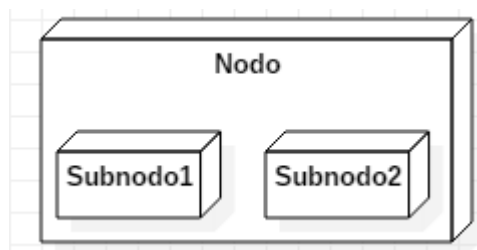


Figura (49) Extraída de Internet UML

Veamos un ejemplo completo

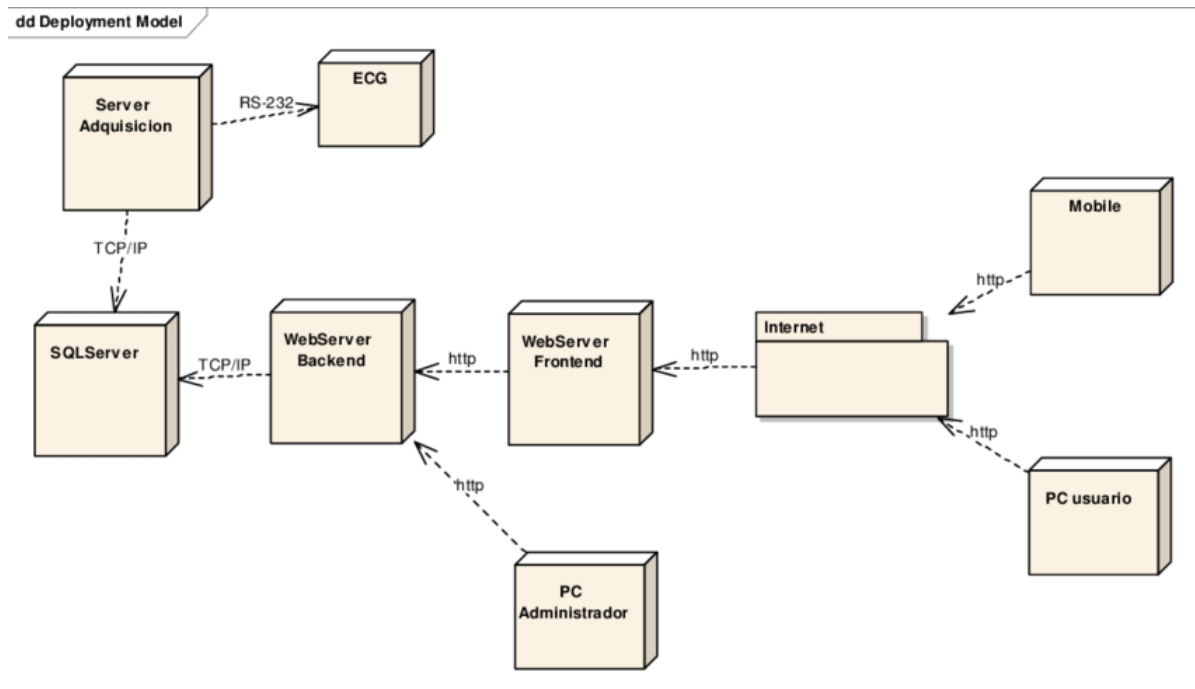


Figura (50) Extraída de Internet UML

Bibliografía

- Ian Sommerville(2011) Ingeniería de Software Novena edición .Ed. Pearson Educación de México, S.A. de C.V. Cap.(4) Cap (23)
- PMI(2008) Fundamentos para la dirección de proyectos(Guía del PMBOK). Cuarta edición.Cap(6))"Gestión del tiempo del proyecto"
- Craig Larman (2003) UML y Patrones.Segunda edición. Ed.Prentice Hall.Cap. (6.13) Diagrama de casos de uso.
- Craig Larman (2003) UML y Patrones.Segunda edición. Ed.Prentice Hall.Cap. (10.4)Guías para el modelado del negocio.
- GilGambarte Formación(2011) Ms Project 2010-2013-2016 .Definición de hitos Disponible en:
- <https://www.youtube.com/watch?v=tg4Ql8tkmBY>
- Manuel.Ciller.es (2017) "Diagrama de componentes" .Disponible en:
- <https://manuel.cillero.es/doc/metrica-3/tecnicas/diagrama-de-componentes/>
- Manuel.Ciller.es (2017)"Diagrama de paquetes" .Disponible en:
- <https://manuel.cillero.es/doc/metrica-3/tecnicas/diagrama-de-paquetes>



Atribución-No Comercial-Sin Derivadas

Se permite descargar esta obra y compartirla, siempre y cuando no sea modificado y/o alterado su contenido, ni se comercialice. Referenciarlo de la siguiente manera:

Universidad Tecnológica Nacional Facultad Regional Córdoba (S/D). Material para la Tecnicatura Universitaria en Programación, modalidad virtual, Córdoba, Argentina.